

An Advanced Persistent Threat Detection Framework Based on Graph Attention Networks with Spectral Feature Refinement

Chun-I Fan, *Senior Member, IEEE*, Ji-Shan Wu, Cheng-Han Shie*, Hsin-Nan Kuo, Bo-Yi Lee

Abstract—Recent studies have proposed APT detection methods based on provenance graphs. However, existing systems face challenges due to large-scale graphs, leading to high memory consumption and limited efficiency. Many datasets provide only preprocessed data without disclosing log collection and attribute mapping, hindering reproducibility. Some approaches rely on commercial EDR or high-quality threat intelligence, limiting real-world adoption.

To address these limitations, this study proposes a Graph Attention Network (GAT)-based APT detection framework with spectral feature refinement. The framework uses Sysmon logs and integrates MITRE ATT&CK technique matching with open-source Sysmon filtering rules to construct Rule-Filtered Provenance Graphs (RFPGs), reducing graph size while preserving security-relevant semantics. Textual and numerical attributes are encoded into unified node and edge representations to model heterogeneous system entities and interactions. A spectrally refined GAT model is then employed for malicious node detection, with an Interval Bias mechanism to adjust detection sensitivity under a controlled precision trade-off.

Experimental results show that the proposed system achieves 98.25% accuracy, 90.58% precision, and a 0.36% false positive rate. Compared with a baseline model, it reduces inference time by up to 10.8% and GPU memory consumption by 11.4%. Model inference takes 0.82 seconds, while the complete pipeline requires approximately 5 seconds.

Index Terms—Advanced Persistent Threat (APT), Machine Learning, Graph Attention Network, Provenance Graph, Threat Intelligence

Chun-I Fan is with the Department of Computer Science and Engineering, the Information Security Research Center, and the Intelligent Electronic Commerce Research Center, National Sun Yat-sen University, Kaohsiung 804201, Taiwan (E-mail: cifan@mail.cse.nsysu.edu.tw).

Ji-Shan Wu is with the Department of Computer Science and Engineering, National Sun Yat-sen University, Kaohsiung 804201, Taiwan (E-mail: boy975263@gmail.com).

Cheng-Han Shie is with the Department of Computer Science and Engineering, National Sun Yat-sen University, Kaohsiung 804201, Taiwan (E-mail: hanhan3927@g-mail.nsysu.edu.tw).

Hsin-Nan Kuo is with the Department of Computer Science and Engineering, National Sun Yat-sen University, Kaohsiung 804201, Taiwan (E-mail: bluedunk@gmail.com).

Bo-Yi Lee is with the National Center for High-performance Computing, Tainan 744094, Taiwan (E-mail: boyi@narlabs.org.tw).

This work was supported in part by the National Science and Technology Council of Taiwan under Grant NSTC 114-2634-F-110-001-MBK, in part by the Information Security Research Center at National Sun Yat-sen University, and in part by the Intelligent Electronic Commerce Research Center from the Featured Areas Research Center Program through the Framework of the Higher Education Sprout Project by the Ministry of Education in Taiwan. The authors would also like to thank Yi-En Chang for proofreading the manuscript.

* Corresponding author

I. INTRODUCTION

Major data breach incidents are often attributed to advanced persistent threats (APTs). In the general definition of APTs, "advanced" refers to the combined use of multiple advanced techniques to gain unauthorized access, and "persistent" refers to the continual and long-term maintenance of such access to specific targets. APT attacks involve complex techniques that enable widespread cyberattacks against targeted organizations. Because the scale of such attacks is usually greater than that of conventional cyberattacks, most APT attacks are initiated by hacker or state-sponsored groups that engage in comprehensive planning, have well-defined objectives, and employ sophisticated technology. In addition, APT attacks are generally conducted in several stages over a long period, with the duration of the attack cycle ranging from weeks to years [1]. APT attacks are mainly divided into three stages. The attacker first detects the Internet configuration and server information of the target. They then install rootkit or malware programs to gain access control permissions. Finally, they use a command-and-control server to issue commands to the breached internal nodes and transmit the stolen data to the malicious domain.

Studies in related fields have attempted to enhance host security by investing considerable resources into the development of various security mechanisms, including antivirus software, encryption technology, vulnerability scanning programs, and patching. However, existing mechanisms cannot address the complex attack patterns of APTs, which continue to evolve [2]. Among the security mechanisms available, APT detection is generally considered the most effective. Motivated by these limitations, recent studies increasingly leverage host audit logs to construct provenance graphs and apply deep representation learning for APT detection, aiming to reduce reliance on hand-crafted rules while improving scalability on massive logs [3]. Early works such as SLEUTH [4] demonstrated that real-time attack scenario reconstruction from audit data is feasible through tag propagation on provenance graphs, laying the foundation for subsequent detection systems. In addition, SHADEWATCHER [5] analyzes system audit records by constructing a knowledge graph and formulating threat discovery as a recommendation-style interaction prediction problem.

The prevailing assertion in related studies is that provenance-based anomaly detection is the most effective method of APT detection [6]. This method collects system logs

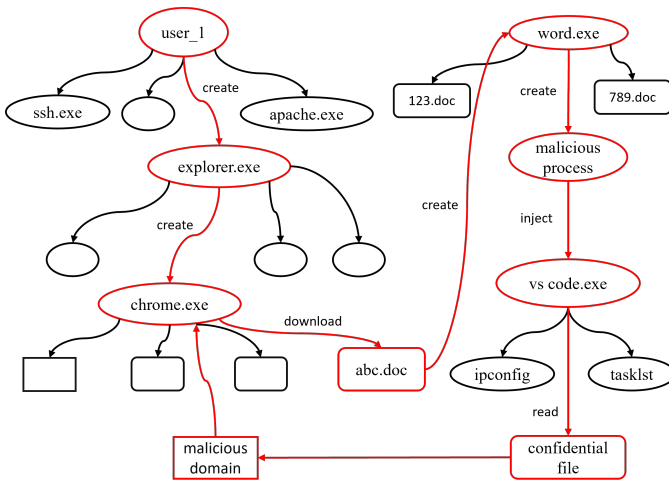


Fig. 1. APT attack provenance graph 1.

to generate provenance data and create provenance graphs.

Recent systems further emphasize practicality, such as lightweight unsupervised detection with time-sliced subgraphs to reduce memory overhead and provide finer-grained localization for investigation [7]. Moreover, it maps anomaly samples to a latent space to determine correlations among these samples. Existing provenance-based anomaly detection approaches often differ in their primary focus: some emphasize graph structure modeling, while others concentrate on contextual or semantic enhancement.

A. Graph-Centric Detection Approaches

Graph-centric methods in provenance-based anomaly detection deploy monitoring software in a host system to capture data flow and control flow logs [8], which are then integrated into provenance data to create a provenance graph. Provenance graphs are a reliable tool for APT detection. They are directed acyclic graphs converted from system logs that depict information flow between system subjects (e.g., programs) and system objects (e.g., system files and Internet ports) [9]. In a directed acyclic graph, nodes represent individual system objects and events, and edges illustrate the relationships between these objects and events.

Fig. 1 presents an APT scenario; the red zone denotes an APT attack path in which the attacker pretends to be a system user named `user_1` and uses `explorer.exe` to download and open `chrome.exe`. Next, the attacker downloads a pre-established malicious file named `abc.doc` to implant a backdoor in the host system and then uses benign software (e.g., Microsoft Word) to execute a malicious process. The malicious file is disguised as `vs code.exe` through an injection technique so that confidential data can be accessed; these data are then transmitted to the malicious domain. In this scenario, provenance graphs are not affected by the cycle of APT attacks and display a rich amount of contextual information in a simple manner, thereby enabling security personnel to immediately determine the current system state, distinguish between benign and malicious events, identify potential attack paths, and prevent misjudgment caused by single events.

Scholars have recently proposed multiple detection methods based on provenance graphs. For example, the UNICORN [6] detector creates a provenance graph for benign activities and compares it with system-generated provenance graphs before and after an attack. When a graph generated by a system deviates from the corresponding benign graph, the detector confirms the occurrence of an ongoing attack. In addition, a graph-sketching technique is implemented in the detector to depict the current system state by creating a histogram, which undergoes breadth-first search so that the intrusion behaviors of the attacker can be captured. Another example is the HOLMES [10] system, which creates graphs for high-level scenarios and maps the information flow of a system to a MITRE ATT&CK [11] kill chain to distinguish between benign and malicious events through threat matching.

To further address the scalability issues inherent in long-running systems, PROGRAPHER [12] leverages a snapshot-based approach. It explicitly targets dependency explosion by splitting streaming logs into temporal-ordered snapshots to reduce computation and memory costs. It then detects anomalous snapshots via whole-graph embedding (Graph2Vec [13]) with sequence learning (TextRCNN [14]), reporting key indicators to narrow down suspicious entities for investigation. Complementary to these detection techniques, systems like Kairos [15] focus on the temporal dynamics of whole-system provenance. It performs attack-agnostic anomaly detection by learning these dynamics and highlighting suspicious activities for subsequent analysis.

Beyond graph-level and snapshot-level detection, recent studies have advanced node-level detection on provenance graphs. ThreaTrace [16] employs GraphSAGE [17] to learn each node's structural role and detect anomalies at fine granularity. FLASH [8] further combines semantic learning with GNN-based structural embeddings, demonstrating scalability and robustness against mimicry attacks. MAGIC [18] leverages masked graph autoencoders for self-supervised detection without requiring attack-labeled training data. R-CAID [19] embeds root cause analysis into provenance-based IDS by precomputing each node's causal origins during graph construction. APT-KGL [20] incorporates heterogeneous threat knowledge into provenance graph learning to further improve detection accuracy.

However, these methods are typically evaluated on curated datasets, such as DARPA TC [21] or DARPA OpTC [22]. Although the log collection in such datasets is based on kernel-level provenance capture, which enables fine-grained tracking of system-level interactions, it often incurs higher overhead compared to event-based logging approaches. In addition, the implementation details of the data collection process are not fully disclosed, making it difficult to assess the deployment cost and reproduce the logging pipeline.

In addition, RapSheet [23] leverages commercial EDR tools and infers causal dependencies among EDR-generated alerts. While system-level provenance and EDR-based approaches can capture richer behavioral details, they often introduce dependencies on specialized functionalities that are not readily available in practical deployment environments. This limitation motivates us to explore whether more accessible yet expressive

logging sources can be used as an alternative for analysis.

However, even with such alternative data sources, managing the sheer volume of data remains a challenge. Because APT attacks commonly last for months or even years, the collection of provenance data over an extensive period continually increases the sizes of provenance graphs. At present, relevant detection methods mainly store provenance graphs in the core system memory to accelerate the detection process; consequently, problems such as reduced detection efficiency and excessive memory usage can occur when the system operates continuously over a long period of time. This situation severely undermines the effectiveness of real-time APT detection systems. Therefore, effective analysis and optimization of provenance graphs is a crucial topic in Graph-centric detection.

B. Context-Enhanced Detection Approaches

Context-enhanced methods map information flow among system subjects and also map these subjects' contextual information to a latent space to perform semantic analysis. Preset definitions or neural networks are then used to distinguish between benign and malicious events. CONAN [2] is a context-enhanced detector that divides an APT attack into multiple phases, namely execution, influence, and external connection. On the basis of such division, feature matching can be performed to detect the attack. The CONAN detector is unique in that instead of creating provenance graphs, it examines the system state to record semantic information and uses a finite state automata-like structure for state conversion and APT detection. This detector has been verified as effective for detecting APT attacks while preventing a continual increase in memory usage over time. Zhu et al. [24] proposed a data compaction method for global semantics and suspicious semantics; by using forensic analysis to restore the original kill chain, this method examines object attributes to automatically determine whether specific events are associated with an attack.

While these context-enhanced approaches effectively capture semantic information, they still face the fundamental scalability challenge common to graph-based methods: the size of a provenance graph increases continuously while the system remains in operation, thereby reducing detection efficiency. Moreover, such methods are mostly knowledge based and therefore cannot detect zero-day attacks. To overcome these limitations, this study developed a graph attention network (GAT [25])-based APT detection system with the following features:

- We propose rule-filtered provenance graphs (RFPGs) to reduce noise and graph scale through ATT&CK-aligned Sysmon rules [26], enabling efficient malicious entity detection with GAT.
- We design a high-dimensional heterogeneous graph encoder for RFPGs to improve the representation quality and facilitate more effective GNN training.
- Our framework incorporates Graph Attention Networks (GATs) [25] with spectral feature refinement layer that preserves salient structural and semantic information in RFPGs while compressing feature dimensionality,

thereby improving computational and memory efficiency with negligible impact on detection effectiveness.

- We develop a practical and efficient APT detection framework that does not rely on commercial EDR solutions or customized security rules. By constructing refined provenance graphs and incorporating spectral feature refinement, the proposed system reduces graph scale and feature dimensionality, thereby improving computational efficiency while maintaining detection effectiveness.

II. GAT-BASED APT DETECTION SYSTEM

This section introduces the detection system developed in this study. Conventional APT detection systems based on provenance data often suffer from excessive memory consumption and degraded detection speed when logs are collected over extended periods. Moreover, such systems typically construct provenance graphs directly from raw system logs, resulting in large-scale graphs in which malicious activities are embedded only within limited subgraphs.

To address these challenges, we develop a GAT-based APT detection with spectral feature refinement. To reduce the scale of provenance graphs derived from raw logs, the system leverages open-source, expert-defined Sysmon rules that are explicitly mapped to MITRE ATT&CK techniques to filter noisy events and retain attack-relevant activities prior to graph construction. These rule-driven graphs are referred to as RFPGs.

The RFPGs are then encoded using a high-dimensional heterogeneous graph encoder to generate expressive feature representations. Finally, GATs with spectral feature refinement are applied to classify nodes, improving detection efficiency while maintaining detection effectiveness.

A. Overview

Fig. 2 presents the framework of the developed system, which involves the following five steps:

- 1) **Attack Generation and Log Collection.** We generate attacks and collect provenance logs in a controlled simulation environment.
- 2) **Graph Construction and Rule Matching.** The collected provenance data are converted into RFPGs through the ATT&CK techniques-aligned rule-matching mechanism.
- 3) **High-dimensional Heterogeneous Graph.** A high-dimensional heterogeneous graph encoder is used to code the RFPGs, which are then stored as heterogeneous data.
- 4) **Model Training.** The heterogeneous data are converted into heterogeneous graphs, and subgraphs of these graphs are used to train a GATs with spectral feature refinement model for node classification.
- 5) **Threat Assessment.** Node classification results obtained from the GATs with spectral feature refinement model are used to detect APT attacks and perform threat assessment.

During the attack generation and log collection phase, we use MITRE CALDERA [27] to simulate APT29 and APT3 attack scenarios, and deploy Sysmon on the victim hosts in each scenario to collect system logs.

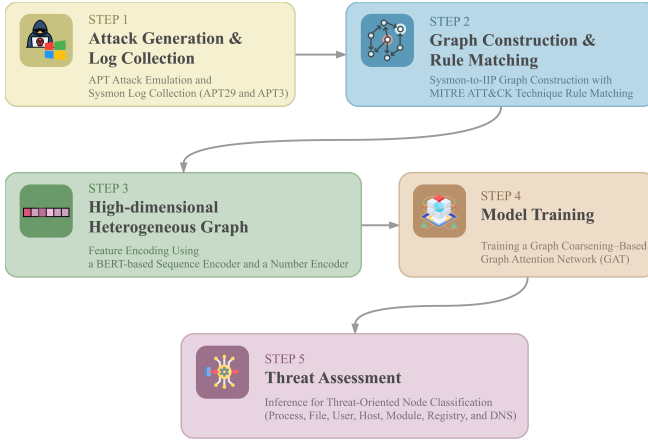


Fig. 2. System framework

In the graph construction and rule-matching phase, RFPGs are built from the collected logs. Specifically, the logs are first processed using a rule-matching mechanism aligned with MITRE ATT&CK techniques to identify and filter Sysmon events corresponding to ATT&CK attack techniques. The matched events are then mapped to their respective technique IDs, and the resulting filtered data are used to construct the RFPGs.

In the next phase, the proposed high-dimensional heterogeneous graph encoder is used to sample and encode data in feature vectors. This encoder converts RFPGs into heterogeneous data for storage, retains the features of the original data, and increases the dimensionality of the converted data. Accordingly, the resulting features comprise more dimensions to more accurately depict the structure of the RFPGs and reflect the significance of each event.

During the model training phase, we incorporate a ChebConv layer [28] into the GAT architecture with spectral feature refinement to reduce the feature dimensionality of RFPGs. Specifically, ChebConv performs localized spectral filtering via Chebyshev polynomial approximation, enabling frequency-domain transformation of graph signals without explicitly computing the graph Fourier transform. This process enhances informative components while suppressing noise. The learned filter parameters further project node features into a lower-dimensional space, achieving feature compression and reducing computational overhead while mitigating excessive information loss.

The constructed model consists of one ChebConv layer followed by two graph attention layers, and is trained on subgraphs extracted from RFPGs. The attention layers adaptively learn the importance weights of neighboring nodes, thereby improving malicious node classification accuracy and overall detection performance.

The final threat assessment phase involves the node classification of the input graph, and the classification results are used to determine whether the host system is under an APT attack.

The following subsections describe the data preprocessing procedure, the proposed model architecture, and the interval

bias mechanism introduced during model training and evaluation. The interval bias mechanism aims to improve the true positive rate (TPR) by adjusting the decision threshold. By allowing a slight increase in the false positive rate (FPR), it effectively reduces the false negative rate while avoiding a significant additional burden on subsequent analysis.

B. Data Preprocessing

Recent studies related to machine learning have mainly focused on neural networks. However, the training results of such models are greatly affected by data quality, feature dimensionality, and data selection. Given the importance of data preprocessing, this section introduces the preprocessing method adopted in this study. Fig. 3 presents all the steps of the data processing procedure, namely data acquisition, provenance graph construction, and feature sampling.

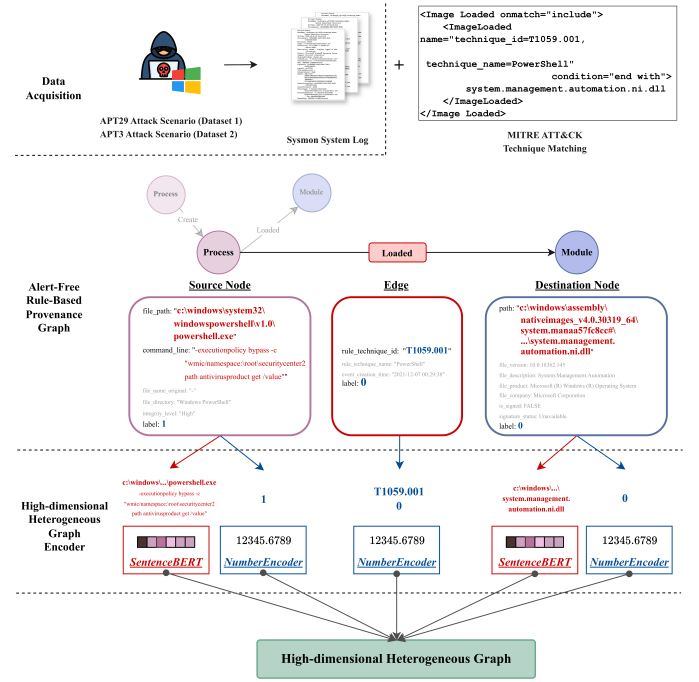


Fig. 3. Data preprocessing

1) *Data Acquisition:* As illustrated in Fig. 3, this study employs a self-constructed dataset for model training and evaluation. The dataset is generated using the MITRE CALDERA platform to simulate both benign and malicious behaviors. The malicious activities incorporate attack techniques adopted by two APT groups, namely APT3 and APT29.

For data collection, Sysmon is adopted as the primary source of system logs. Sysmon is a widely deployed system monitoring tool in enterprise Windows environments and continuously records host-level system activities and security-related events. In this study, Sysmon is utilized to monitor fundamental system behaviors and generate event logs. The collected host logs are transmitted through an event collection mechanism to a designated analysis engine for centralized storage and management.

```
Image loaded:
UtcTime: 2020-05-02 02:57:15.988
ProcessGuid: {47ab858c-e18b-5eac-b103
-000000000400}
ProcessId: 78
Image: c:\windows\system32\windowpowershell\
v1.0\powershell.exe
ImageLoaded: C:\Windows\assembly\
NativeImages_v4.0.30319_64\System.
Manaa57fc8cc#\
fa56adb2347ed3a045d8f7471018af68\
System.Management.Automation.ni.dll
Hashes: SHA1=7238
FFDB8A14F8F572DA42896F38FA332132E8B7
Signed: False
Signature:
SignatureStatus: Unavailable
```

Listing 1. Sysmon Event ID 7 Triggered by PowerShell Execution

```
<ImageLoaded onmatch="include">
  <ImageLoaded name="technique_id=T1059
    .001, technique_name=PowerShell"
    condition="end with">
    system.management.automation.ni.dll
  </ImageLoaded>
</ImageLoaded>
```

Listing 2. Matching rule for MITRE ATT&CK T1059.001(PowerShell).

The detailed procedures of attack emulation, log collection architecture, and dataset construction and storage will be presented in Section III-A and Section III-B, respectively.

2) *Rule-Filtered Provenance Graphs (RFPGs)*: After completing log collection, a data preprocessing phase is conducted to construct the proposed Rule-Filtered Provenance Graphs (RFPGs). The design of RFPGs is inspired by the IIP graph introduced in RapSheet [23], which leverages security alerts to identify and filter threat-related provenance data and annotates them with corresponding MITRE ATT&CK techniques.

Unlike RapSheet, which relies on alerts generated by commercial EDR solutions, the proposed RFPGs utilize publicly available expert-designed matching rules to filter events. These rules identify system events exhibiting potential threat characteristics, thereby eliminating large volumes of benign or irrelevant log entries. This filtering process reduces the overall graph size, enabling the detection model to more efficiently learn critical threat behaviors and achieve faster convergence.

The adopted matching rules are derived from the Sysmon Modular [29] open-source project, which provides a curated Sysmon configuration file. The rules defined in this configuration were developed by multiple cybersecurity experts based on threat analysis. In total, 766 rules correspond to 104 MITRE ATT&CK techniques. When a Sysmon event satisfies a predefined rule, the corresponding ATT&CK technique ID and technique name are attached to the event record.

For high-risk attack categories such as credential access, privilege escalation, and persistence, Sysmon Modular provides multiple dedicated matching rules, demonstrating that Sysmon event types can effectively capture these high-impact attack behaviors.

For example, Listing 1 presents a Sysmon Event ID 7 record. This type of event corresponds to an Image Loaded

event, which logs module loading activities and records information such as the process responsible for loading the module, the file hash, and digital signature details. According to the matching rule defined in Listing 2, when the Sysmon log entry shown in Listing 1 indicates that the loaded module name ends with system.management.automation.ni.dll, the rule is triggered and recorded. The corresponding event is then annotated with MITRE ATT&CK technique T1059.001.

In addition, regarding the selection of Sysmon event types, the collected events were systematically organized and filtered. The selected event types are summarized in Table I, based on which the data model illustrated in Fig. 4 was constructed. In this data model, edges represent not only the causal relationships between events but also the corresponding MITRE ATT&CK technique IDs identified through the proposed matching rules.

As shown in Table II, in the RFPG graph model, only Process and Host nodes serve as source nodes. In contrast, destination nodes may correspond to different entity types, including Process, File, Host, and Module. The complete RFPG graph is constructed by correlating events through the unique identifiers of the source nodes together with their associated timestamps.

Finally, regarding the design of node and edge attributes, each edge is annotated with the corresponding MITRE ATT&CK technique ID derived from the mapping rules. For node attributes, Sysmon events provide extensive and detailed information associated with each entity (as summarized in Appendix C). In principle, multiple fields could be utilized as node features. However, incorporating all available attributes would substantially increase the feature dimensionality, potentially hindering model convergence during training.

To mitigate the convergence issue caused by high-dimensional representations, we select one representative attribute for each node type as its primary feature, as shown in Table III. These selected attributes capture the most discriminative information for each entity type. Nevertheless, the proposed framework remains extensible. Additional attributes can be incorporated by concatenating them with the original feature and encoding the combined representation into a fixed-dimensional vector for subsequent graph neural network processing.

TABLE I
SYSMON EVENT IDS AND DESCRIPTIONS USED IN THIS WORK.

No.	Event ID	Description
1	Event ID 1	Process Creation
2	Event ID 2	File Creation Time Changed
3	Event ID 3	Network Connection Detected
4	Event ID 5	Process Terminated
5	Event ID 7	Image Loaded
6	Event ID 10	Process Access
7	Event ID 11	File Create
8	Event ID 12	Registry Event (Object Create and Delete)
9	Event ID 13	Registry Event (Value Set)
10	Event ID 22	DNSEvent

3) *High-dimensional Heterogeneous Graph Encoder*: Regarding feature sampling, this study applied two encoders, namely a number encoder and a sequence encoder, to code

TABLE II
RELATION-CENTRIC MAPPING FROM SYSMON LOGS TO
PROVENANCE-GRAPH RELATIONS.

Source Node	Edge	Target Node	Event ID
<i>Process-to-Resource Relations</i>			
Process	Create	Process	1
Process	Access	Process	10
Process	RequestAccess	Process	10
Process	Create	File	11
Process	Modify	File	2
Process	Create	Registry	12
Process	Delete	Registry	12
Process	Modify	Registry	13
Process	Load	Module	7
Process	Connect	Host	3
Process	Query	DNS	22
<i>User-to-Resource Relations</i>			
User	Create	Process	1
User	Terminate	Process	5
User	Load	Module	7
User	Connect	Host	3

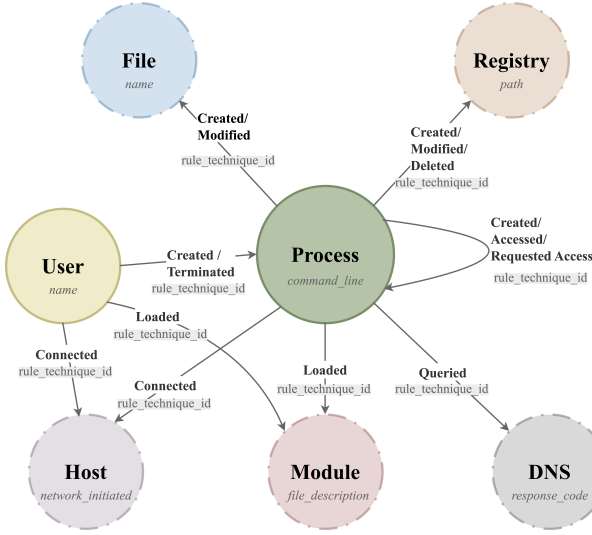


Fig. 4. Rule-Filtered Provenance Graphs(RFPGs) Data Model

graphical features. The sequence encoder comprises the All-MiniLM-L6-v2 [30] model, which is a pretrained sentence transformer based on natural language processing in Sentence-BERT. This model exhibits high data processing speed and is therefore suitable for handling graph problems with large numbers of nodes.

The All-MiniLM-L6-v2 model recognizes semantic information in RFPGs nodes (e.g., Process, File, Host, Module, User, and DNS) and maps the nodes in the original graph space to a latent space, where nodes with similar semantic information are located close to one another to reflect node features with higher dimensionality.

Regarding edge features, the developed number encoder directly translates the technique ID number of each edge into a floating point number for storage. For example, the technique ID number T1055.001 is converted into 1055.001 after the translation process. The number encoder also codes each node label in the RFPGs.

Next, the RFPGs is coded by the high-dimensional het-

erogeneous graph encoder to obtain a heterogeneous graph structure. The encoded data are stored as heterogeneous data in a database and may be converted into heterogeneous graphs when necessary.

In a heterogeneous graph, each different category of edges describes a distinct relationship between multiple categories of nodes. A heterogeneous graph is defined as a graph G that comprises a set of nodes $V = \{v_1, v_2, \dots, v_n\}$ and edges $E = \{e_1, e_2, \dots, e_m\}$.

Each node or edge in the graph corresponds to a specific category, and T_v and T_e are used to denote the node category set and edge category set, respectively. A heterogeneous graph has two mapping functions that individually match each node or edge to its corresponding category. An encoded graph structure is stored as a tensor.

When training the GAT model or performing APT detection, the constructed heterogeneous RFGP is first converted into a homogeneous graph representation, as the original GAT architecture is designed for homogeneous graphs with a single node and edge type. Subsequently, isolated nodes and self-loops are removed, and the resulting graph is fed into the GAT model.

C. Graph Attention Networks (GATs) with Spectral Feature Refinement

Fig. 5 presents the framework of the GAT model spectral feature refinement developed in this study. First, the pre-processed RFGP are used as the model input data. A dropout layer is deployed to prevent overfitting due to sample insufficiency or excessive training. This layer uses the Bernoulli distribution to randomly zero several elements of node features at a probability of 60%, thereby masking some of the neurons in the model during training. Accordingly, the masked neurons are not updated during back propagation. This approach has been verified to effectively resolve problems concerning input normalization and model overfitting. The output of the dropout layer is then used as the input for the ChebConv layer [28], which uses the node features, edge index, and edge attributes of the input graph to perform convolution operation. The hidden channels of the input vary with respect to the number of node features F . The ChebConv layer effectively enhances the training and verification speed of the overall model without severely compromising model accuracy.

Because the developed system uses subgraphs of RFPGs to perform model training, the model should be capable of detecting unknown events. Relationships between model layers are nonlinear; thus, the output of the ChebConv layer is calculated using a rectified linear unit (ReLU) activation function. This function increases the level of nonlinearity between model layers, exhibits a high calculation speed, and prevents overfitting and gradient vanishing. The ReLU activation function is expressed as

$$\text{ReLU}(X) = X' = \max(0, X), \quad (1)$$

where X represents the input of the ReLU function and is equal to $\{(x_1, x_2, x_3, \dots, x_n)\}$. In addition, x_1 to x_n denote the input features of the ReLU function, n is the number of

TABLE III
NODE ATTRIBUTES WITH EXAMPLES.

Node	Attribute	Example
Process	command_line	"c:\users\public\updater.exe" -server http://192.168.117.12:9999 -group red
File	name	c:\windows\prefetch\fontdrvhost.exe-e2768e90.pf
User	name	nt authority\system
Host	network_initiated	network initiated
Module	file_description	OLE32 Extensions for Win32
Registry	path	HKLM\System\CurrentControlSet\Services\CredentialEnrollmentManagerUserSvc_18b7cf
DNS	response_code	response

output channels, and X' is the output of the ReLU function. According to the ReLU equation, the output of this function is either 0 or a positive number.

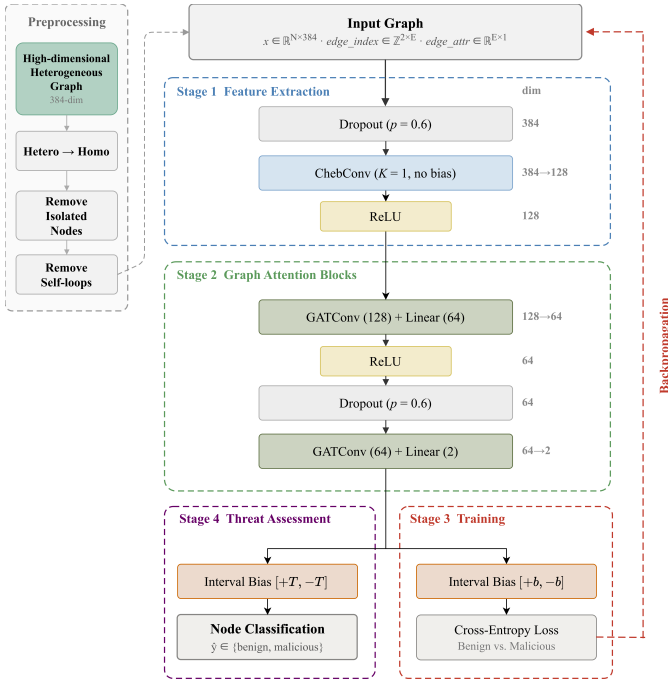


Fig. 5. Model Structure

Next, the input graph structure undergoes treatment from the attention mechanism of the GAT layer. Because this mechanism assigns distinct weights to nodes according to variations in these nodes' neighbors, it enables higher model accuracy than does the ChebConv layer. Related studies have demonstrated that GAT models exhibit higher adaptability than do GCN models [25]. The developed detection system comprises two GAT layers. GAT Layer 1 contains 128 input data and 64 input neurons, and GAT Layer 2 contains 64 input data and two output neurons. These two layers are connected by a ReLU function.

The experiment performed in this study revealed that the ratio of positive samples to negative samples in the collected provenance data was highly unbalanced. Because the ChebConv layer was designed on the basis of commonly used graph datasets (e.g., Cora and Citeseer), a low TPR was obtained when this layer was used to process the collected provenance data, indicating that the established model was ineffective in correctly identifying malicious samples. The main purpose of

APT detection systems is to detect potential attack events so that security personnel can verify the threat levels of these events. Consequently, a low TPR is highly problematic because it indicates that numerous attacks might have been undetected, preventing security personnel from implementing immediate measures against these attacks. The FPR of a model may increase when the TPR increases. The FPR represents the proportion of benign events that are mistakenly identified as malicious events by the model. A high FPR affects the judgement of security personnel. In this study, the design of the developed system prioritized the functionality of detecting malicious events.

After multiple rounds of experimentation, this study discovered that malicious nodes with a specific interval of output values were frequently mistakenly identified as benign nodes. Accordingly, an interval bias layer was designed to adjust output values through the following equation:

$$\mathbf{z}' = \mathbf{z} + [-b, b], \quad (2)$$

where $\mathbf{z} = [x_0, x_1]$ represents the benign scores and malicious scores, respectively, related to the features output by GAT Layer 2, and where b is the interval bias, which is adjusted according to the attributes of different datasets. During model training, the interval bias layer increases benign scores and reduces malicious scores in model outputs. Accordingly, malicious nodes with benign scores lower than the interval bias are reclassified as benign nodes. By contrast, malicious nodes that are mistakenly identified as benign nodes exhibit increased benign scores and reduced malicious scores. This score adjustment approach increases the model loss and enables greater weight adjustment during back propagation. Accordingly, the established model tends to recognize nodes with a specific interval of output values as malicious, and the output benign scores and malicious scores are reduced and increased, respectively, compared with models without an interval bias layer.

D. Interval Bias in Model Assessment

The interval bias layer introduced previously increases the TPR of the established model. However, model assessment does not entail the calculation of the loss function or back propagation. Accordingly, this study developed another interval bias function specifically for model assessment to increase the APT attack detection rate. This function is expressed as follows:

Algorithm 1 Forward Pass

Require: Node features $\mathbf{X} \in \mathbb{R}^{N \times 384}$, adjacency $\mathbf{A}$, edge features $\mathbf{E} \in \mathbb{R}^{M \times 1}$
Ensure: Logits $\mathbf{Z} \in \mathbb{R}^{N \times 2}$

- 1: **function** FORWARDPASS($\mathbf{X}, \mathbf{A}, \mathbf{E}$)
- 2: $\mathbf{H} \leftarrow \text{Dropout}(\mathbf{X}, p = 0.6)$
- 3: $\mathbf{H} \leftarrow \text{ReLU}(\text{ChebConv}_{K=1, out=128}(\mathbf{H}, \mathbf{A}))$
- 4: $\mathbf{H} \leftarrow \text{GAT}_1(\mathbf{H}, \mathbf{A}, \mathbf{E}) + \text{Linear}_{1^{128 \rightarrow 64}}(\mathbf{H})$
- 5: $\mathbf{H} \leftarrow \text{ReLU}(\mathbf{H})$
- 6: $\mathbf{H} \leftarrow \text{Dropout}(\mathbf{H}, p = 0.6)$
- 7: $\mathbf{Z} \leftarrow \text{GAT}_2(\mathbf{H}, \mathbf{A}, \mathbf{E}) + \text{Linear}_2^{(64 \rightarrow 2)}(\mathbf{H})$
- 8: **return** $\mathbf{Z}$
- 9: **end function**

Algorithm 2 Training with Interval Bias

Require: Graph $G = (V, \mathcal{E})$ with labels $\mathbf{Y}$, bias b , learning rate η , epochs T_{ep}
Ensure: Trained parameters θ

- 1: $\mathbf{X} \leftarrow \text{SentenceBERT}(V)$
- 2: $\mathbf{E} \leftarrow \text{NumberEncoder}(\mathcal{E})$
- 3: $\mathbf{A} \leftarrow \text{Adjacency}(\mathcal{E})$
- 4: $\mathcal{V}_{tr}, \mathcal{V}_{te} \leftarrow \text{StratifiedSplit}(V, r_{\text{train}} = 0.2)$
- 5: Initialize θ ; set $\mathbf{b} = [b, -b]$
- 6: **for** $t = 1$ **to** T_{ep} **do**
- 7: $\mathbf{Z} \leftarrow \text{ForwardPass}(\mathbf{X}, \mathbf{A}, \mathbf{E}; \theta)$
- 8: $\mathbf{Z} \leftarrow \mathbf{Z} + \mathbf{1}_N \mathbf{b}^\top$ ▷ Interval Bias, $\mathbf{1}_N \in \mathbb{R}^N$
- 9: $\mathcal{L} \leftarrow \text{CrossEntropy}(\mathbf{Z}[\mathcal{V}_{tr}], \mathbf{Y}[\mathcal{V}_{tr}])$
- 10: $\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}$
- 11: **end for**
- 12: **return** θ

$$x'_0 = \begin{cases} x_0 - T, & x_0 > -T, \\ x_0, & \text{otherwise,} \end{cases} \quad x'_1 = \begin{cases} x_1 + T, & x_1 < T, \\ x_1, & \text{otherwise,} \end{cases} \quad (3)$$

where x_0, x_1 represent the benign scores and malicious scores, respectively, T denotes the interval of output values that frequently causes the model to mistakenly identify malicious nodes as benign nodes. The interval bias function adjusts the output values of the established model. Specifically, the benign score of a node is reduced when it is greater than $-T$, and the malicious score of a node is increased when it is lower than T .

III. EVALUATION

This section introduces the experimental configuration of this study and analyzes the experimental results. In Section III-A, the dataset and attack scenarios explored in this study are discussed. In Section III-C, the experimental configuration and results are described.

A. Attack Generation

1) *Limitations of Existing Public APT Datasets:* Although multiple APT attack detection systems have been proposed, reliable and public APT attack datasets are lacking, let alone

Algorithm 3 Node-Level Assessment with Interval Bias

Require: Trained parameters θ , features $\mathbf{X}, \mathbf{A}, \mathbf{E}$, test nodes $\mathcal{V}_{\text{test}}$, bias b
Ensure: Predictions $\hat{\mathbf{Y}}$, malicious scores $\mathbf{S}$

- 1: $\mathbf{Z} \leftarrow \text{ForwardPass}(\mathbf{X}, \mathbf{A}, \mathbf{E}; \theta)$
- 2: **if** $b > 0$ **then** ▷ Interval Bias
- 3: $\mathbf{Z}_{:,1} \leftarrow \mathbf{Z}_{:,1} + b \cdot \mathbb{1}[\mathbf{Z}_{:,1} < b]$
- 4: $\mathbf{Z}_{:,0} \leftarrow \mathbf{Z}_{:,0} - b \cdot \mathbb{1}[\mathbf{Z}_{:,0} > -b]$
- 5: **end if**
- 6: $\hat{\mathbf{Y}} \leftarrow \arg \max_{c \in \{0,1\}} \mathbf{Z}[\mathcal{V}_{\text{test}}, c]$
- 7: $\mathbf{S} \leftarrow \text{Softmax}(\mathbf{Z}[\mathcal{V}_{\text{test}}])_{:,1}$
- 8: **return** $\hat{\mathbf{Y}}, \mathbf{S}$

a provenance graph dataset related to APT attacks. Related studies have used a variety of datasets to develop detection systems. For example, Gbadebo et al. [31] assessed the performance of the OAML method by simulating seven types of cyberattack scenarios. However, because APT attack incidents are usually highly complex, simulated attacks cannot effectively replicate actual attack scenarios. Consequently, differences exist between the dataset used by Gbadebo et al. and real-world APT attacks.

Han et al. [6] used the dataset in the StreamSpot [32] system to assess the performance of the UNICORN detector. StreamSpot is a clustering-based anomaly detection system that collects system logs and converts them into streaming heterogeneous graphs. Established in 2016, the StreamSpot dataset contains five benign scenarios and one malicious scenario. However, these scenarios are not designed specifically to represent APT attacks. Consequently, the data in the dataset cannot reflect real-world APT attacks. Xiong et al. [2] used the dataset provided by DARPA TC to develop the CONAN detector. This dataset originated from the Transparent Computing Program of the Defense Advanced Research Projects Agency. In this program, simulated cyberattacks and security measures were implanted into multiple famous research institutions in the United States, and the event streams generated from these attacks were analyzed and recorded. Compared with other datasets, this dataset more accurately depicts APT attack scenarios. However, it is still less authentic than the MITRE CALDERA, which incorporates real-world APT attack scenarios.

The proposed RFPGs in this study are constructed based on raw Sysmon logs. However, most existing public APT datasets, such as the widely used DARPA TC dataset, do not provide complete raw Sysmon logs. Instead, they often rely on expert-curated analyses or high-quality labels generated by commercial EDR solutions as input to detection models. Although such data facilitate improved model performance, comparable high-quality annotations are typically unavailable in real-world deployment scenarios.

To better reflect practical conditions, we adopt the official Windows system monitoring tool, Sysmon, to collect event logs. Sysmon is widely available and can be readily deployed on enterprise hosts, making it suitable for realistic evaluation.

2) *Attack Generation*: Based on this design principle, we simulate four attack scenarios to generate training and evaluation datasets, including two scenarios derived from the MITRE ATT&CK framework, one data exfiltration scenario, and one RapSheet-based data exfiltration scenario executed via MITRE CALDERA.

Attack scenarios were implemented in two simulation environments. In Simulation Environment 1, the APT29 attack scenario in MITRE ATT&CK and the data exfiltration scenario were used. APT29 was originally initiated by a hacker group in Russia. Using flexible strategies to attack targets around the globe, this group tends to adopt the most sophisticated APT techniques. The kill chain of APT29 analyzed in this study was based on RapSheet. As shown in Fig. 6, this attack scenario includes multiple techniques, including Registry Run Keys (T1060), Process Injection (T1055), and Defense-Evasion Execution (T1085). These techniques collectively correspond to the persistence, privilege escalation, lateral movement, and defense evasion of APT attacks.

To increase the number and diversity of malicious samples in the dataset, the data exfiltration scenario was also implemented in Simulation Environment 1. Data exfiltration is a common technique for data theft and unauthorized movement. Specifically, loopholes in Windows Update are utilized to invade a target host, and techniques including T1055 and T1086 are used to execute multiple malicious PowerShell programs and determine the location of confidential data. A camouflage program is then used to steal these data, which are compressed and transmitted to a command-and-control server through techniques including T1074.001 and T1041.

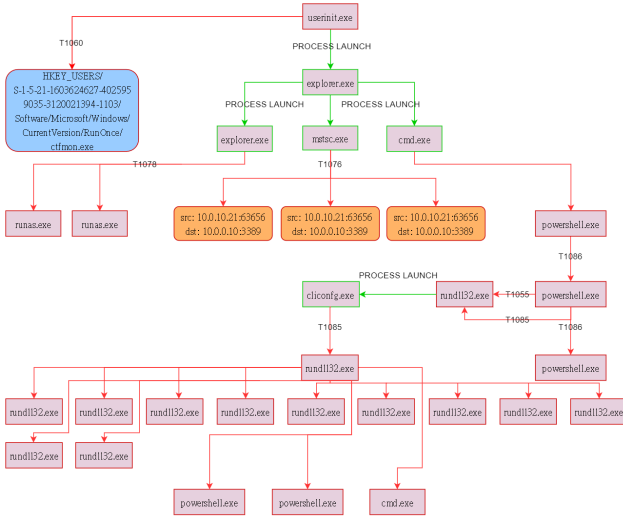


Fig. 6. APT29 IIP Graph

In Simulation Environment 2, the APT3 attack scenario in MITRE ATT&CK and the MITRE CALDERA data exfiltration scenario provided in RapSheet were implemented. APT3 was originally initiated by a hacker group from China and has since been the subject of a considerable amount of research; all techniques and strategies used in APT3 have been labeled

in the MITRE ATT&CK guideline. The kill chain of APT3 analyzed in this study was based on RapSheet. First, System Service Discovery (T1007) and Remote Desktop Protocol (T1076) are used to attack the target by utilizing loopholes in Windows Remote Desktop. These techniques collectively correspond to malicious program execution, lateral movement, and defense evasion in APT attacks.

The CALDERA data exfiltration scenario was designed on the basis of RapSheet. CALDERA is an emulation platform based on MITRE ATT&CK that provides a command-and-control server interface for connection with malicious software. Therefore, this platform can be used to conveniently simulate common cyberattack scenarios. In the CALDERA data exfiltration scenario, social media are used to prompt users to install malicious programs. This scenario can be employed to simulate zero-day attacks to prevent detection systems from determining the IIP. Techniques including System Owner/User Discovery (T1033), Remote File Copying (T1105), confidential data access, and lateral movement are used to infiltrate the target host and transmit confidential data to the command-and-control server.

B. Dataset

This study simulated the aforementioned attack scenarios in a virtual machine with Windows 10 and used Sysmon to collect system logs. Sysmon is a Windows system program designed to monitor and record system activities and to address the shortcomings of Windows in assessing system events. The collected user logs were then transmitted to the ELK stack [33] for event analysis and matching. The ELK stack is a log analysis and management system that comprises the Elasticsearch, Logstash, and Kibana projects. Finally, the Neo4j [34] graph database was used to visualize the matched event data and generate an RFPGs accordingly. The RFPGs generated from Simulation Environments 1 and 2 were named Dataset 1 and Dataset 2, respectively.

Table IV summarizes the properties of Dataset 1 and Dataset 2. Each dataset consists of seven types of nodes (as shown in Table III) and fifteen types of events (as listed in Table II). The proportion of malicious nodes is 4.7% and 5.1%, respectively, indicating a clear class imbalance in both datasets.

In addition, approximately 88.79% and 88.92% of the edges are matched to MITRE ATT&CK techniques. However, the fact that an event is mapped to an ATT&CK technique does not necessarily imply malicious behavior. Therefore, the GAT model is required to learn the relational patterns between benign and malicious activities to achieve effective detection.

In addition, Table V presents the distribution of MITRE ATT&CK techniques matched to the edges within malicious-labeled subgraphs in Dataset 1 and Dataset 2. Among them, T1059.001 (PowerShell) accounts for the largest proportion.

C. Experiment Result

1) *Experimental Settings*: The developed system was used to detect attack scenarios simulated in Windows 10. Linux Ubuntu 24.04.2 LTS was used to perform model training and assessment. The experimental host comprised the AMD Ryzen

TABLE IV
PROPERTIES OF DATASETS

	Dataset 1	Dataset 2
<i>Nodes</i>		
Total	873	935
Malicious	41 (~4.7%)	48 (~5.1%)
Benign	832	887
<i>Edges</i>		
Total	2,158	2,185
Mapped Technique	1,916 (~88.79%)	1,943 (~88.92%)
Unmapped Technique	242	242

TABLE V
MITRE ATT&CK TECHNIQUE DISTRIBUTION ON MALICIOUS EDGES

	Dataset 1		Dataset 2	
Technique	#	%	#	%
T1059.001 PowerShell	41	51.8	57	52.8
T1036 Masquerading	18	22.2	18	16.7
T1085 Rundll32	9	11.1	7	6.5
T1105 Ingress Tool Transfer	–	–	12	11.1
T1076 Remote Desktop	3	3.7	4	3.7
T1073 DLL Side-Loading	2	2.5	2	1.9
T1078 Valid Accounts	2	2.5	–	–
T1140 Deobfuscate	2	2.5	–	–
T1055 Process Injection	1	1.2	1	0.9
T1060 Registry Run Keys	1	1.2	–	–
T1083 File Discovery	1	1.2	–	–
T1070 Indicator Removal	–	–	1	0.9
T1204 User Execution	–	–	1	0.9
T1047 WMI	–	–	1	0.9
T1003 Credential Dumping	–	–	1	0.9
T1049 Net. Conn. Discovery	–	–	1	0.9
T1082 Sys. Info. Discovery	–	–	1	0.9
T1033 Sys. Owner Discovery	–	–	1	0.9
Total	81	100	108	100

9 9900X processor and 64 GB of RAM, and NVIDIA GeForce RTX 5070ti was used to accelerate the model training speed.

Table VI summarizes the hyperparameter settings used in the experiments, which are also applied in the subsequent ablation studies. If different hyperparameter configurations are adopted in specific experiments, they are explicitly noted in the corresponding figures or tables. It is worth noting that the interval bias mechanism is not applied in the standard experimental setting and is evaluated only in the interval bias-related ablation and comparative experiments.

2) *Baseline Experimental Results:* Tables VIII and IX present the baseline inference results without applying the interval bias mechanism. The model was trained on 20% of the data and evaluated on the remaining 80%. Each experiment was repeated 10 times with different random seeds, and the results are reported as mean $\pm$ standard deviation along with the 95% confidence intervals.

The results show that the Precision reaches 90.58% and 93.14% for Datasets 1 and 2, respectively, while the FPR is 3.6% and 2.9%. This corresponds to approximately 2–3 benign nodes being misclassified as malicious. However, the TPR is 69.35% and 69.85%, respectively. Considering that 80% of

TABLE VI
HYPERPARAMETERS

Hyperparameter	Value
Optimizer	Adam
Learning rate	0.001
Weight decay	5×10^{-4}
Epochs	5,000
Batch size	Full-batch (entire graph)
Dropout rate	0.6
Loss function	Cross-entropy
Data split strategy	RandomNodeSplit(split="train_rest")
Training samples per class	20
Train / Test ratio	20% / 80%
Interval bias (Train)	[0, 0]
Interval bias (Assessment)	[0, 0]

the malicious nodes are included in the test set, approximately 11–13 malicious nodes remain undetected.

Based on these observations, the Section III-C3 evaluates the interval bias mechanism. By appropriately adjusting the decision interval, the model aims to reduce false negatives at the cost of a tolerable decrease in Precision, thereby improving the overall detection capability for malicious nodes.

Table VII presents the memory usage and generation time of the RFPGs for Datasets 1 and 2. The RFPG of Dataset 1 was generated in 4.18 seconds and consumed 230.6 MB of GPU memory. The RFPG of Dataset 2 was generated in 4.04 seconds and consumed 230.6 MB of main memory.

Notably, the majority of memory usage is attributed to loading the pre-trained All-MiniLM-L6-v2 model, while the graph data itself occupies approximately 96 MB.

TABLE VII
DATASET STATISTICS AND GRAPH CONSTRUCTION COST

	Dataset 1	Dataset 2
Nodes / Edges	873 / 2,157	935 / 2,184
Construction Time	4.18s $\pm$ 0.21	4.04s $\pm$ 0.15
GPU Peak Memory	230.6 MB	230.6 MB

TABLE VIII
PERFORMANCE EVALUATION OF THE GNN MODEL – DATASET 1 (10 RUNS WITH 95% CONFIDENCE INTERVALS)

Metric	Mean	Std Dev	95% CI
<i>Detection Performance</i>			
Accuracy	0.9825	0.0066	[0.9778, 0.9873]
TPR (Recall)	0.6935	0.0965	[0.6245, 0.7626]
Precision	0.9058	0.0944	[0.8383, 0.9733]
FPR	0.0036	0.0038	[0.0009, 0.0063]
F1 Score	0.7822	0.0826	[0.7231, 0.8413]
ROC-AUC	0.9616	0.0218	[0.9460, 0.9772]
PR-AUC	0.8123	0.0878	[0.7495, 0.8750]
<i>Computational Cost</i>			
Training Time (sec)	10.37	0.08	[10.32, 10.43]
Inference Time (ms)	0.82	0.02	[0.80, 0.83]
Train GPU Peak (MB)	25.94	0.10	[25.87, 26.01]
Infer GPU Peak (MB)	22.80	0.00	[22.80, 22.80]

3) *Experimental Results under Different Interval Bias Values:* Since the developed system exhibits relatively low TPR

TABLE IX
PERFORMANCE EVALUATION OF THE GNN MODEL – DATASET 2 (10 RUNS WITH 95% CONFIDENCE INTERVALS)

Metric	Mean	Std Dev	95% CI
<i>Detection Performance</i>			
Accuracy	0.9810	0.0063	[0.9765, 0.9856]
TPR (Recall)	0.6985	0.1159	[0.6156, 0.7813]
Precision	0.9314	0.0370	[0.9050, 0.9579]
FPR	0.0029	0.0017	[0.0017, 0.0041]
F1 Score	0.7914	0.0774	[0.7360, 0.8467]
ROC-AUC	0.9800	0.0088	[0.9737, 0.9863]
PR-AUC	0.8700	0.0289	[0.8493, 0.8906]
<i>Computational Cost</i>			
Training Time (sec)	10.52	0.10	[10.44, 10.59]
Inference Time (ms)	0.82	0.02	[0.80, 0.83]
Train GPU Peak (MB)	26.10	0.14	[25.99, 26.20]
Infer GPU Peak (MB)	22.97	0.22	[22.81, 23.12]

without the interval bias layer, we further evaluate its effectiveness by applying multiple bias values. Fig. 7 and Fig. 8 present the results for Dataset 1 and Dataset 2 under different interval bias settings.

In the figures, the X-axis represents the interval bias T applied during inference, while the Y-axis denotes the interval bias b introduced during training. The performance transition from the lower-left corner (no bias applied) to the upper-right corner (with $T = b = 3$) illustrates the impact of different bias combinations. All results are reported as mean $\pm$ standard deviation over 10 independent runs.

The results indicate that for both Dataset 1 and Dataset 2, increasing the bias to 1 ($T = b = 1$) leads to a noticeable improvement in TPR with only a slight reduction in Precision. In particular, for Dataset 2, when the bias is increased to 2 ($T = b = 2$), the Precision remains around 80%. Overall, these findings suggest that the interval bias mechanism effectively improves TPR while maintaining an acceptable level of Precision, thereby reducing false negatives.

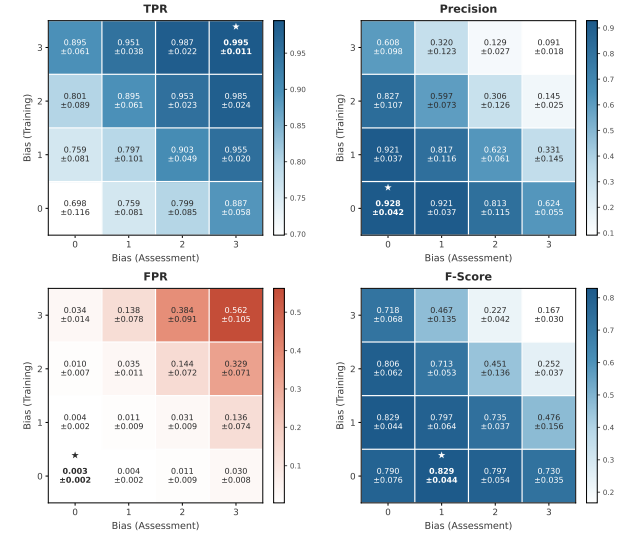


Fig. 8. Heatmap of Performance Metrics under Different Interval Bias Settings (Dataset 2)

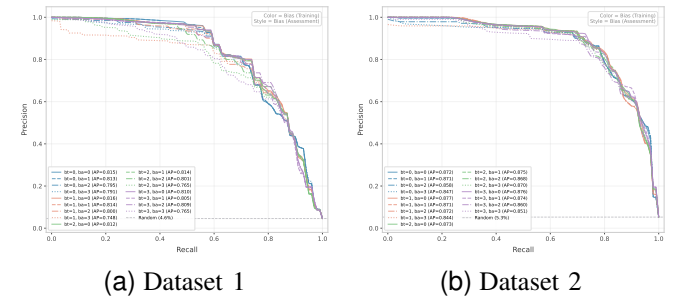


Fig. 9. Precision-Recall Curves under Different Interval Bias Settings

4) *Experimental Results under ChebConv Layer and GAT Layer:* Fig. 10 and Fig. 11 present the experimental computation cost and performance on Dataset 1 and Dataset 2 after incorporating Spectral Feature Refinement via a ChebConv layer. Under a setting of 15,000 epochs, replacing the feature extraction component with ChebConv instead of a GAT layer reduces the training and inference time by approximately 15% and 8.5%, respectively, on Dataset 1. Meanwhile, the peak GPU memory usage is reduced by approximately 17.1% and 11.4%.

In terms of detection performance, Precision shows a slight improvement. Although TPR exhibits a minor decrease across both datasets, it can be further adjusted through the interval bias mechanism. Overall, the model maintains comparable detection performance while achieving reductions in training time, inference time, and memory consumption.

5) *Experimental Results under Different Chebyshev Filter Orders K :* The results in Fig. 12 indicate that varying the Chebyshev filter order (K) does not lead to significant differences across performance metrics. The experimental results show that, for both Dataset 1 and Dataset 2, the overall model performance remains comparable under different K settings, suggesting that the model is relatively robust to changes in the Chebyshev polynomial order.

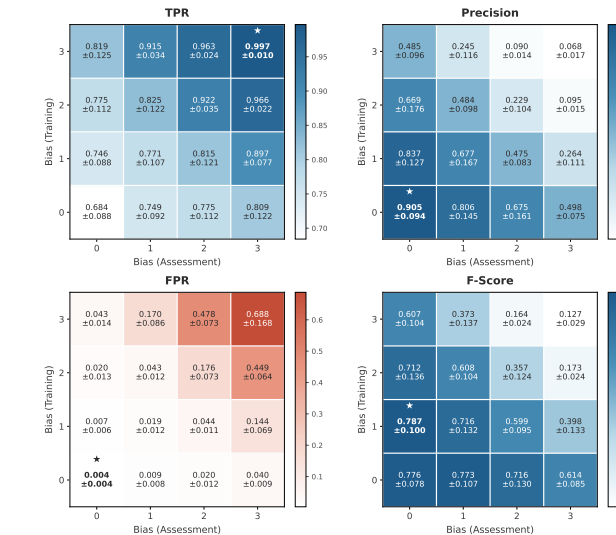


Fig. 7. Heatmap of Performance Metrics under Different Interval Bias Settings (Dataset 1)



Fig. 10. Performance Metrics under ChebConv and GAT Layers

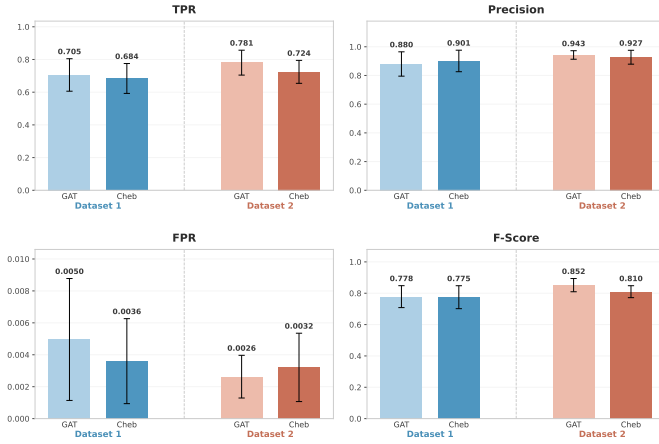


Fig. 11. Computation Metrics under ChebConv and GAT Layers

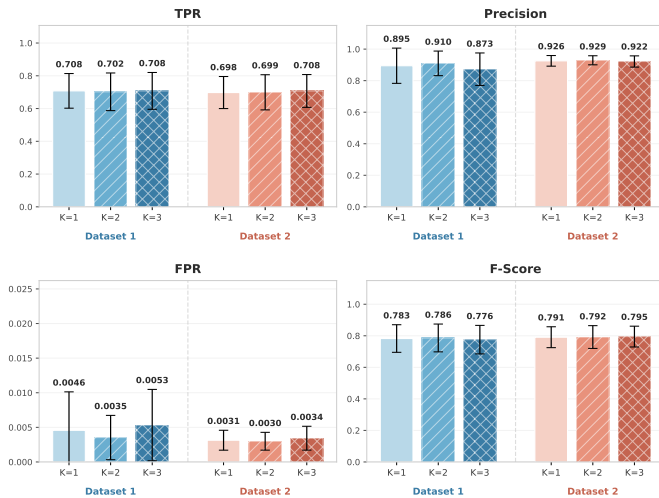


Fig. 12. Performance Metrics under Different Chebyshev Filter Orders K

6) *Experimental Results With and Without MITRE ATT&CK Technique IDs:* The experimental results in Fig. 13 indicate that removing the Technique ID attributes from edges does not significantly affect the overall model performance. This suggests that the model primarily relies on the structural properties of the graph and the relational patterns between nodes, rather than directly depending on Technique IDs to determine whether a node is malicious or benign.

Although the experimental results show that Technique IDs have a limited direct impact on detection performance, they still play an important role in the proposed RFGP framework. The construction of RFGPs is based on an ATT&CK technique-aligned rule-matching mechanism. While the involvement of a node in specific Technique IDs does not solely determine its maliciousness, the behavioral context and relational interactions are more critical factors. Moreover, Technique IDs serve as a standardized language for describing attack behaviors. After detection, the matched Technique IDs can assist security analysts in rapidly understanding and investigating the attack context, thereby enhancing the practical applicability of the system.

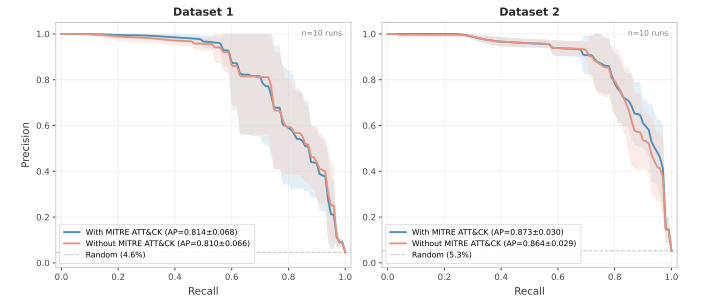


Fig. 13. Performance Metrics With and Without MITRE ATT&CK Technique IDs

D. Discussion & Limitation

As summarized in Table X, existing APT detection studies adopt different benchmark or self-generated datasets, including StreamSpot, DARPA TC, DARPA OpTC, and MITRE CALDERA. Their collection mechanisms also vary, ranging from system-call auditing and CDM-based kernel provenance capture to CamFlow/CamQuery and commercial EDR telemetry. However, many prior studies provide only limited details regarding the end-to-end pipeline from log collection to provenance graph construction, which makes reproduction and practical deployment challenging. In contrast, our proposed method relies on Sysmon logs generated from MITRE CALDERA scenarios and provides an end-to-end reproducible collection and detection pipeline.

Based on the above considerations, this study adopts Sysmon as the log collection tool and integrates open-source MITRE ATT&CK technique matching and filtering rules to construct a reproducible and practically feasible detection process. In addition, we employ GNNs as the modeling approach. GNNs have been widely used for analyzing provenance graphs, and their effectiveness has been validated in

TABLE X
COMPARISON OF DATASETS AND COLLECTION PIPELINES

Study	Dataset	Collection Mechanism	Reproducibility
UNICORN [6], IPG [35]	StreamSpot	System calls	Limited
CONAN [2], PROGRAPHER [12], MAGIC [18]	DARPA TC	CDM-based kernel provenance	Limited
FLASH [8], TAPAS [36]	DARPA TC / OpTC	CDM-based kernel provenance	Limited
OAML [31]	MITRE CALDERA	CamFlow / CamQuery	Partial
RapSheet [23]	MITRE CALDERA	Commercial EDR	Partial
Proposed Method	MITRE CALDERA	Sysmon	End-to-end

Limited indicates that the end-to-end pipeline from log collection to graph construction is not fully disclosed or readily reproducible.

numerous studies, which also ensures comparability with existing research.

Moreover, most publicly available datasets do not provide raw logs in their original form. Instead, they are typically pre-filtered to prevent excessive dataset size and improve usability. While such processing facilitates fair evaluation, these curated datasets may not fully reflect the characteristics of real-world APT attacks.

To collect more original Sysmon logs, this study refers to the data collection methodology and attack scenarios proposed by RapSheet and simulates real APT attack events documented in the MITRE ATT&CK database. The network attack techniques adopted in these events can be labeled using the corresponding MITRE ATT&CK technique IDs, which represent commonly used techniques in APT attacks. Furthermore, based on the original scenarios, this study also incorporates a data exfiltration scenario, which is a common scenario in real-world APT attacks.

Although RapSheet also implements APT3 and APT29 attacks using the MITRE CALDERA framework (OAML does not specify its implementation details), the attack scenarios, log collection methods, and scales differ across studies. Therefore, even though the performance metrics appear comparable according to experimental results, it is not possible to ensure consistent evaluation standards due to the inability to reproduce their attack scenarios or obtain their datasets. Nevertheless, the proposed method in this study has greater potential for practical application and implementation.

Moreover, although the current study does not support other operating systems, compared with other approaches, the proposed method presents a solution with extensibility potential.

Finally, to increase the TPR, various interval bias values were used to train the GAT model. However, the FPR was increased alongside the TPR, and this compromise could not be overcome.

IV. RELATED WORKS

Although APT detection is a relatively new research topic, multiple studies regarding relevant detection techniques have already been published. This section introduces several essential techniques proposed in the present study by describing the practicality, accuracy, and robustness of these techniques.

A. OAML

Proposed by Gbadebo et al. [31], online adaptive metric learning (OAML) is a detection method based on provenance

graphs. This method uses metric learning to classify benign and malicious events in a latent feature space. Metric learning is a deep learning framework that converts an original feature space into a latent feature space. Gbadebo et al. employed CamFlow and other system recording tools to capture provenance data, which were then converted into provenance graphs. To apply these graphs in the training of metric learning models, the researchers referenced the node2vec technique proposed by Grover et al [37]. This technique combines the advantages of breadth-first search and depth-first search to convert original graphs into vectors while retaining the features of those original graphs. In addition, to perform model training with the converted features, Gbadebo et al. proposed an online metric learning method, which employs an adaptive-bound triplet loss function to estimate the distance between each sample and an anchor in the latent space. This function is expressed as follows, where x_t, x_t^+, x_t^- denote the anchor, positive sample, and negative sample, respectively.

$$S = \{(x_t, x_t^+, x_t^-)\}_{t=1}^T \quad (4)$$

The triplet loss function automatically adjusts the upper and lower bounds suitable for model training. Multiple training iterations are performed to shorten the distances between similar samples and increase those between dissimilar samples. The same study also implemented an online triplet mining method to acquire samples. In that method, predefined rules are used to automatically identify valid triplets, which are then sorted into batches so that model training can be performed. Because metric learning primarily focuses on the grouping of similar samples into the same category, it is more suitable for APT detection than are conventional machine learning methods. In addition to the ability of the OAML method to detect unknown attacks, Gbadebo et al. [31] also verified that this method was 12% more accurate than were conventional machine learning methods.

B. RapSheet

Developed by Hassan et al. [23], RapSheet is the first detection method to combine the MITRE ATT&CK framework with tactical provenance graphs (TPGs). This method uses the Symantec endpoint detection and response tool to collect provenance data in a host system. This tool is primarily deployed in enterprise settings and uses the rule-matching mechanism of MITRE ATT&CK, which matches individual system events with corresponding ATT&CK techniques to convert a provenance graph into an IIP graph. As shown

in Fig. 14, each edge in an IIP graph carries a technique ID, enabling the graph to retain high-dimensional features. RapSheet further employs causality-preserved reduction and graph pruning to simplify IIP graphs, and combines them into a single-chain TPG. Threat alerts in the TPG are then ranked along the kill chain: events that match the ATT&CK kill chain in chronological order receive higher scores than those appearing in a random sequence, and the resulting threat score determines whether the host system is under attack.

Despite these contributions, RapSheet has notable limitations. First, as a knowledge-based method, it cannot detect zero-day or unknown attacks that fall outside predefined rules. Second, it may be susceptible to adaptive attacks that deliberately evade rule matching. Third, most relevant to the present work, RapSheet relies on Symantec’s commercial EDR to generate IIP graphs, which limits reproducibility and raises deployment barriers for organizations without access to such proprietary tools. The proposed framework draws inspiration from RapSheet’s IIP graph concept but addresses these limitations: it replaces commercial EDR dependency with publicly available Sysmon Modular rules to construct RFGs, making the pipeline fully reproducible, and employs a learnable GAT model that can capture relational patterns beyond predefined rule-based scoring.

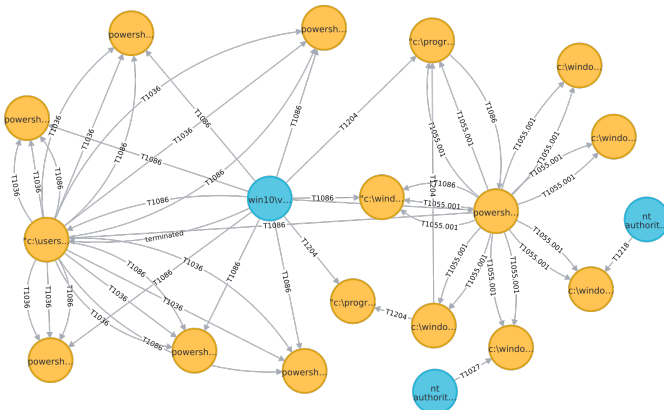


Fig. 14. Initial Infection Point Graph

C. FLASH

Proposed by Rehman et al. [8], FLASH combines semantic learning with GNN-based structural embeddings for intrusion detection on provenance graphs. It demonstrates scalability and robustness against mimicry attacks on DARPA TC datasets. However, FLASH relies on the pre-processed provenance data provided by DARPA TC without disclosing the log collection procedure, and its evaluation does not consider the practical constraints of real-world log acquisition. In contrast, our method explicitly addresses the log collection and filtering pipeline using open-source Sysmon rules.

D. MAGIC

Proposed by Jia et al. [18], MAGIC employs masked graph autoencoders for self-supervised APT detection on provenance

graphs, eliminating the need for attack-labeled training data. By reconstructing masked node and edge features, MAGIC learns normal behavioral patterns and flags deviations as anomalies. While this self-supervised paradigm reduces labeling effort, it is evaluated exclusively on DARPA TC datasets and does not address the reproducibility of the log collection pipeline. The proposed framework differs in adopting a supervised approach with an open-source, end-to-end reproducible workflow from log collection to detection.

E. TAPAS

Proposed by Zhang et al. [36], TAPAS is an online APT detection framework that addresses the challenge that APT attacks require long-term monitoring over months, leading to rapidly growing provenance graphs and consequently high storage and computational overhead. To tackle this issue, TAPAS focuses on the backbone of the provenance graph, which is typically large-scale but sparse. It reduces graph complexity in both spatial and temporal dimensions by employing a stacked LSTM-GRU [38] model and a task-guided backbone graph partitioning algorithm. This algorithm enables real-time identification of active subgraphs as detection targets, thereby reducing structural redundancy across time and space. Similar to many prior works, TAPAS is evaluated on widely used DARPA TC and OpTC datasets. Experimental results show that TAPAS can reduce storage requirements by up to $1806\times$, achieve an accuracy of 99.99%, and attain an average detection time of 12.78 seconds per GB of audit data.

However, similar to many existing approaches, TAPAS relies on DARPA benchmark datasets and does not address the reproducibility of the end-to-end log collection and graph construction pipeline. Moreover, by transforming provenance graphs into sequential representations using LSTM-GRU models, TAPAS primarily captures process-level behavioral patterns, which may lead to the loss of fine-grained structural information, particularly at the edge level. In contrast, our GNN-based approach directly operates on graph structures, enabling more fine-grained modeling of inter-entity relationships.

V. CONCLUSION AND FUTURE WORKS

This study develops an APT attack detection system based on a GAT with spectral feature refinement. The system leverages Sysmon event logs together with an open-source, expert-defined MITRE ATT&CK technique matching mechanism to generate RFPGs, whose scale is smaller than that of the original provenance graphs due to rule-based filtering. Through the high-dimensional heterogeneous graph encoder proposed in this study, an efficient heterogeneous graph representation that preserves essential graph features is constructed. Finally, the system employs the spectrally refined GAT model to identify malicious nodes.

The experimental results validate the effectiveness of the developed detection system. The model achieved an accuracy of $98.25\% \pm 0.66\%$, a precision of $90.58\% \pm 9.44\%$, a FPR of $0.36\% \pm 0.38\%$, and a TPR of $69.35\% \pm 9.65\%$. Furthermore, through the Interval Bias mechanism, the detection performance can be further improved at the cost of

an acceptable reduction in accuracy. In terms of efficiency, the entire detection process requires only 5 seconds when executed on a general consumer-grade GPU. Finally, compared with existing methods, the proposed approach does not rely on commercial EDR solutions or other hard-to-obtain input data, demonstrating its practical applicability and scalability.

In future work, we will continue to prioritize practicality and real-world deployability. First, we plan to extend the proposed framework to other operating systems by leveraging expert-defined logging rules. For example, Linux auditd rules [39], which cover a wide range of security-relevant activities, can serve as a solid foundation for constructing provenance graphs in Linux environments.

In addition, we will explore higher-level or semantically refined provenance graph construction methods. For instance, FG-CIBGC [40] employs In-Context Learning to extract fused semantics from multiple log sources—including audit logs, application logs, and network traffic—and generates behavior graphs with higher-level semantic meaning. Such approaches can enhance the semantic richness of graph structures and improve the representation of attack behaviors.

Finally, as indicated by the scaling laws [41] and emergent abilities [42] of large language models (LLMs), increasing model scale leads to stronger capabilities in semantic understanding and cross-source information integration. Future work may leverage LLMs to perform semantic extraction and integration across heterogeneous log data and threat intelligence, thereby enriching the semantic representation of provenance graphs.

To mitigate the potential hallucination risks of LLMs, GNNs can be incorporated to perform structure-aware correlation analysis and validation, ensuring that inference results are constrained by actual event relationships. The integration of semantic models and structural models can improve the interpretability and robustness of analysis results, while addressing the challenges of multi-source and long-context data integration.

At the same time, emerging work such as ProvGuard [43] highlights that graph-driven approaches may also be vulnerable to graph manipulation attacks, suggesting that robustness against adversarial graph modifications remains an important direction for future research.

REFERENCES

- [1] D. Sullivan, "Beyond the hype: advanced persistent threats," *Advanced Persistent Threats and Real Time Threat Management. The Essential Series*. Realtime Publishers, 2011.
- [2] C. Xiong, T. Zhu, W. Dong, L. Ruan, R. Yang, Y. Chen, Y. Cheng, S. Cheng, and X. Chen, "Conan: A practical real-time apt detection system with high accuracy and efficiency," *IEEE Transactions on Dependable and Secure Computing*, 2020.
- [3] B. Xu, Y. Gong, X. Geng, Y. Li, C. Dong, S. Liu, Y. Liu, B. Jiang, and Z. Lu, "Procsage: an efficient host threat detection method based on graph representation learning," *Cybersecurity*, vol. 7, p. 51, 2024.
- [4] M. N. Hossain, S. M. Milajerdi, J. Wang, S. Sultana, R. Gjomemo, R. Sekar, S. Stoller, and V. Venkatakrishnan, "SLEUTH: Real-time attack scenario reconstruction from COTS audit data," in *Proceedings of the 26th USENIX Security Symposium*, 2017, pp. 487–504.
- [5] J. Zengy, X. Wang, J. Liu, Y. Chen, Z. Liang, T.-S. Chua, and Z. L. Chua, "Shadewatcher: Recommendation-guided cyber threat analysis using system audit records," in *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2022, pp. 489–506.
- [6] X. Han, T. Pasquier, A. Bates, J. Mickens, and M. Seltzer, "Unicorn: Runtime provenance-based detector for advanced persistent threats," in *Proceedings of the Network and Distributed Systems Security Symposium (NDSS)*. Internet Society, 2020, pp. 1–18.
- [7] K. Jiang, Z. Gao, S. Zhang, and F. Zou, "Sylph: An unsupervised apt detection system based on the provenance graph," *Information*, vol. 16, no. 7, p. 566, jul 2025.
- [8] M. U. Rehman, H. Ahmadi, and W. U. Hassan, "FLASH: A comprehensive approach to intrusion detection via provenance graph representation learning," in *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*, 2024, pp. 3552–3570.
- [9] M. A. Inam, Y. Chen, A. Goyal, J. Liu, J. Mink, N. Michael, S. Gaur, A. Bates, and W. U. Hassan, "SoK: History is a vast early warning system: Auditing the provenance of system intrusions," in *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*, 2023, pp. 2620–2638.
- [10] S. M. Milajerdi, R. Gjomemo, B. Eshete, R. Sekar, and V. Venkatakrishnan, "Holmes: real-time apt detection through correlation of suspicious information flows," in *2019 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2019, pp. 1137–1152.
- [11] "MITRE ATT&CK," <https://attack.mitre.org/>, 2024, accessed: 2025-05-01.
- [12] F. Yang, J. Xu, C. Xiong, Z. Li, and K. Zhang, "{PROGRAPHER}: An anomaly detection system based on provenance graph embedding," in *32nd USENIX Security Symposium (USENIX Security 23)*, 2023, pp. 4355–4372.
- [13] A. Narayanan, M. Chandramohan, R. Venkatesan, L. Chen, Y. Liu, and S. Jaiswal, Graph2vec: Learning Distributed Representations of Graphs. [Online]. Available: <http://arxiv.org/abs/1707.05005>
- [14] S. Lai, L. Xu, K. Liu, and J. Zhao, "Recurrent convolutional neural networks for text classification," in *Proceedings of the Twenty-Ninth AAAI Conference on Artificial Intelligence*, ser. AAAI'15. AAAI Press, 2015, pp. 2267–2273.
- [15] Z. Cheng, Q. Lv, J. Liang, Y. Wang, D. Sun, T. Pasquier, and X. Han, "Kairos: Practical intrusion detection and investigation using whole-system provenance," in *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2024, pp. 3533–3551.
- [16] S. Wang, Z. Wang, T. Zhou, H. Sun, X. Yin, D. Han, H. Zhang, X. Shi, and J. Yang, "ThreaTrace: Detecting and tracing host-based threats in node level through provenance graph learning," *IEEE Transactions on Information Forensics and Security*, vol. 17, pp. 3972–3987, 2022.
- [17] W. L. Hamilton, R. Ying, and J. Leskovec, Inductive Representation Learning on Large Graphs. [Online]. Available: <http://arxiv.org/abs/1706.02216>
- [18] Z. Jia, Y. Xiong, Y. Nan, Y. Zhang, J. Zhao, and M. Wen, "MAGIC: Detecting advanced persistent threats via masked graph representation learning," in *Proceedings of the 33rd USENIX Security Symposium*, 2024, pp. 6617–6634.
- [19] A. Goyal, G. Wang, and A. Bates, "R-CAID: Embedding root cause analysis within provenance-based intrusion detection," in *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*, 2024, pp. 3515–3532.
- [20] T. Chen, C. Dong, M. Lv, Q. Song, H. Liu, T. Zhu, K. Xu, L. Chen, S. Ji, and Y. Fan, "Apt-kgl: An intelligent apt detection system based on threat knowledge and heterogeneous provenance graph learning," *IEEE Transactions on Dependable and Secure Computing*, 2022.
- [21] "Darpa-i2o/Transparent-Computing," DARPA I2O. [Online]. Available: <https://github.com/darpa-i2o/Transparent-Computing>
- [22] "FiveDirections/OpTC-data," Five Directions. [Online]. Available: <https://github.com/FiveDirections/OpTC-data>
- [23] W. U. Hassan, A. Bates, and D. Marino, "Tactical provenance analysis for endpoint detection and response systems," in *2020 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2020, pp. 1172–1189.
- [24] T. Zhu, J. Wang, L. Ruan, C. Xiong, J. Yu, Y. Li, Y. Chen, M. Lv, and T. Chen, "General, efficient, and real-time data compaction strategy for apt forensic analysis," *IEEE Transactions on Information Forensics and Security*, 2021.
- [25] P. Velićković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *Proceedings of the 6th International Conference on Learning Representations (ICLR)*, 2018.
- [26] markruss. Sysmon - Sysinternals. [Online]. Available: <https://learn.microsoft.com/en-us/sysinternals/downloads/sysmon>
- [27] MITRE Corporation, "MITRE Caldera," <https://caldera.mitre.org/>, accessed: 2026-02-23.
- [28] M. Defferrard, X. Bresson, and P. Vandergheynst, "Convolutional neural networks on graphs with fast localized spectral filtering," in *Advances in Neural Information Processing Systems (NIPS)*, 2016, pp. 3844–3852.

- [29] O. Hartong, "Sysmon-modular: A repository of sysmon configuration modules," <https://github.com/olafhartong/sysmon-modular>, 2026, accessed: 2026-02-18.
- [30] N. Reimers and I. Gurevych, "Making monolingual sentence embeddings multilingual using knowledge distillation," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 11 2020. [Online]. Available: <https://arxiv.org/abs/2004.09813>
- [31] G. Ayoade, K. A. Akbar, P. Sahoo, Y. Gao, A. Agarwal, K. Jee, L. Khan, and A. Singhal, "Evolving advanced persistent threat detection using provenance graph and metric learning," in *2020 IEEE Conference on Communications and Network Security (CNS)*. IEEE, 2020, pp. 1–9.
- [32] E. A. Manzoor, S. Momeni, V. N. Venkatakrishnan, and L. Akoglu. Fast Memory-efficient Anomaly Detection in Streaming Heterogeneous Graphs. [Online]. Available: <http://arxiv.org/abs/1602.04844>
- [33] Elastic Stack: (ELK) Elasticsearch, Kibana & Logstash. Elastic. [Online]. Available: <https://www.elastic.co/elastic-stack>
- [34] Neo4j. [Online]. Available: <https://neo4j.com/>
- [35] Z. Li, X. Cheng, L. Sun, J. Zhang, and B. Chen, "A hierarchical approach for advanced persistent threat detection with attention-based graph neural networks," *Security and Communication Networks*, vol. 2021, 2021.
- [36] B. Zhang, Y. Gao, C. Yu, B. Kuang, Z. Zhang, H. Kim, and A. Fu, "TAPAS: An efficient online APT detection with task-guided process provenance graph segmentation and analysis," in *34th USENIX Security Symposium (USENIX Security 25)*, 2025, pp. 607–624.
- [37] A. Grover and J. Leskovec, "node2vec: Scalable feature learning for networks," in *Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining*, 2016, pp. 855–864.
- [38] M. Pan, A. Liu, Y. Yu, P. Wang, J. Li, Y. Liu, S. Lv, and H. Zhu, "Radar HRRP Target Recognition Model Based on a Stacked CNN–Bi-RNN With Attention Mechanism," vol. 60, pp. 1–14. [Online]. Available: <https://ieeexplore.ieee.org/document/9352965>
- [39] F. Roth, "Auditd: Best practice auditd configuration rules," <https://github.com/Neo23x0/auditd>, 2026, accessed: 2026-02-24.
- [40] Z. Ni, P. Fan, S. Dai, B. Zhang, H. Wan, and X. Zhao, "Fg-cibgc: A unified framework for fine-grained and class-incremental behavior graph classification," in *Proceedings of the ACM on Web Conference 2025*, ser. WWW '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 1166–1181. [Online]. Available: <https://doi.org/10.1145/3696410.3714960>
- [41] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei. Scaling Laws for Neural Language Models. [Online]. Available: <http://arxiv.org/abs/2001.08361>
- [42] J. Wei, Y. Tay, R. Bommasani, C. Raffel, B. Zoph, S. Borgeaud, D. Yogatama, M. Bosma, D. Zhou, D. Metzler, E. H. Chi, T. Hashimoto, O. Vinyals, P. Liang, J. Dean, and W. Fedus. Emergent Abilities of Large Language Models. [Online]. Available: <http://arxiv.org/abs/2206.07682>
- [43] A. Sang, L. Yang, L. Zhou, C. Zhou, J. Jia, and H. Yang, "ProvGuard: Logic-Aware Multi-View Contrastive Learning for Robust and Efficient Host Threat Detection," in *Proceedings of the ACM Web Conference 2026*, ser. WWW '26. Association for Computing Machinery, 2026, pp. 2776–2787. [Online]. Available: <https://dl.acm.org/doi/10.1145/3774904.3792269>